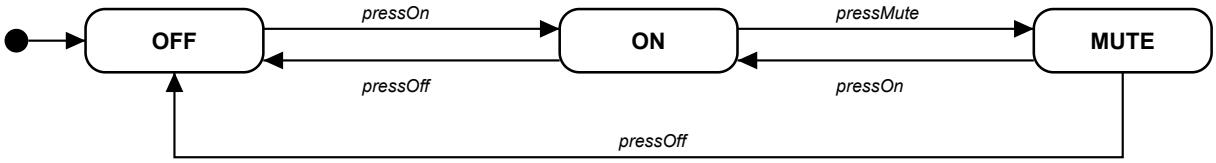
09 State

Behavioral

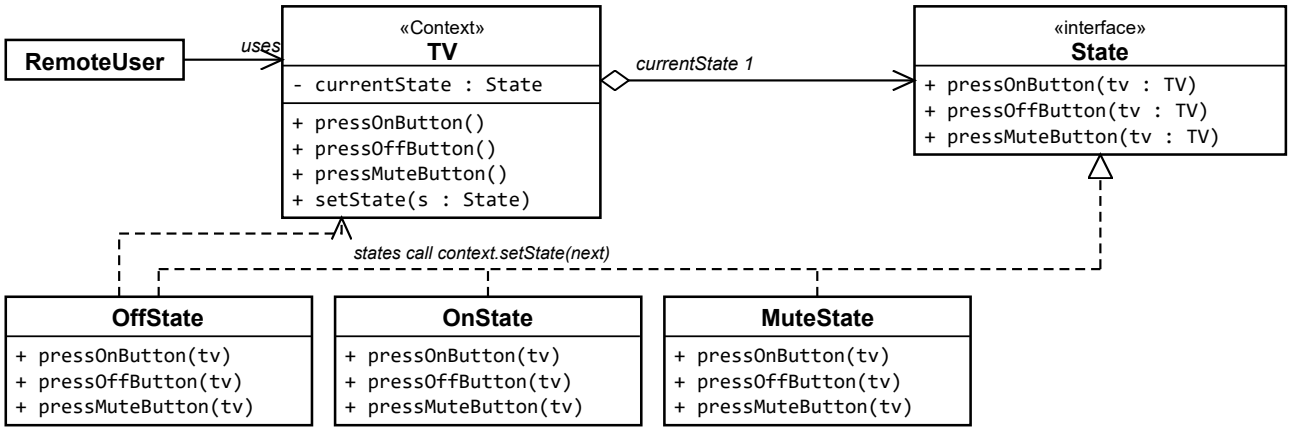
Intent (GoF)	Allow an object to **alter its behaviour when its internal state changes**. The object will appear to change its class.
Use when	An object has a **finite set of well-defined states**; the **same action gives different results depending on the current state**; code is full of `if (state == ...)`/switch blocks repeated in many methods; transitions follow rules.
Trigger words	“can move through the following states”, “transitions to”, “behaviour depends on its current state”, “operates in different states: Idle, Scanning, Payment, Completed”, “draw a state diagram”, order NEW → PAID → SHIPPED → DELIVERED, “behaviour changes via flags/conditions”.
Participants	**Context** (holds `currentState`, delegates requests, has `setState()`) · **State** interface (operations all states support) · **ConcreteStates** (state-specific behaviour, trigger transitions by calling `context.setState(next)`) · **Client** (creates/configures the context).
Design steps	(1) **Draw the state machine first** (states + events) → (2) state table: **State + Event → Behaviour + Next State** → (3) State interface = one method per event → (4) one class per state → (5) Context delegates: `currentState.pressButton(this)`.

TV remote (lecture) — state diagram, state table, UML



State	Event	Behaviour	Next
OFF	ON	turn TV on	ON
OFF	OFF / MUTE	no state change	OFF
ON	ON	no state change	ON
ON	MUTE	sound off	MUTE
ON	OFF	turn TV off	OFF
MUTE	ON	sound back on	ON
MUTE	OFF	turn TV off	OFF

Same button ≠ same behaviour: result depends on the **current state**. A state is more than a value: it decides what the object can do, how it responds, and where it can go next.



```
public interface State {
    void pressOnButton(TV tv);
    void pressOffButton(TV tv);
    void pressMuteButton(TV tv);
}
public class TV {                                   // CONTEXT
    private State currentState = new OffState();    // initial
    public void setState(State s) { currentState = s; }
    public void pressOnButton()   {
currentState.pressOnButton(this); }
    public void pressOffButton()  {
currentState.pressOffButton(this); }
    public void pressMuteButton() {
currentState.pressMuteButton(this); }
}
// BEFORE (bad): if (state==OFF) {...} else if (state==ON) {...}
... in every method
```

```
public class OffState implements State {
    public void pressOnButton(TV tv)  { System.out.println("TV
on"); tv.setState(new OnState()); }
    public void pressOffButton(TV tv) {
System.out.println("Already off"); }
    public void pressMuteButton(TV tv){ System.out.println("TV
is off"); }
}
public class OnState implements State {
    public void pressOnButton(TV tv)  {
System.out.println("Already on"); }
    public void pressOffButton(TV tv) { System.out.println("TV
off"); tv.setState(new OffState()); }
    public void pressMuteButton(TV tv){
System.out.println("Muted"); tv.setState(new MuteState()); }
}
public class MuteState implements State {
    public void pressOnButton(TV tv)  {
System.out.println("Unmuted"); tv.setState(new OnState()); }
    public void pressOffButton(TV tv) { System.out.println("TV
off"); tv.setState(new OffState()); }
    public void pressMuteButton(TV tv){
System.out.println("Already muted"); }
}
TV tv = new TV(); tv.pressOnButton(); tv.pressMuteButton();
tv.pressOffButton();
```

Delegation replaces conditional logic: the context no longer asks “which state am I?”, it says “current state, please handle this”. Other lecture examples: **vending machine** (IdleState, HasOneDollarState, OutOfStockState; `insertDollar/ejectMoney/dispense`; states can be shared **Singletons**), **Person health** (Healthy, Sick, Critical; `work/rest/exercise`; transitions inside state classes).

Advantages / disadvantages / challenge → mechanism

Advantages	Disadvantages	Challenge → mechanism
Removes long `if/switch` on state; cleaner code	**More classes** (one per state)	State-class explosion → use only when states are finite and behaviour differs a lot; few simple states ⇒ plain if is fine
Each state’s behaviour **encapsulated** in one class (SRP)	Transitions spread across classes; harder to see the whole machine	Keep a state diagram + state table as documentation; centralize allowed transitions or validate in `setState()`
Open/Closed: add a new state = new class	Adding an *event* changes every state class	Use an abstract base state with default “invalid action” methods; states override only valid ones
Explicit, valid transitions; easier testing per state	Overkill for 2–3 trivial states	Many state objects → stateless states as Singletons / shared instances

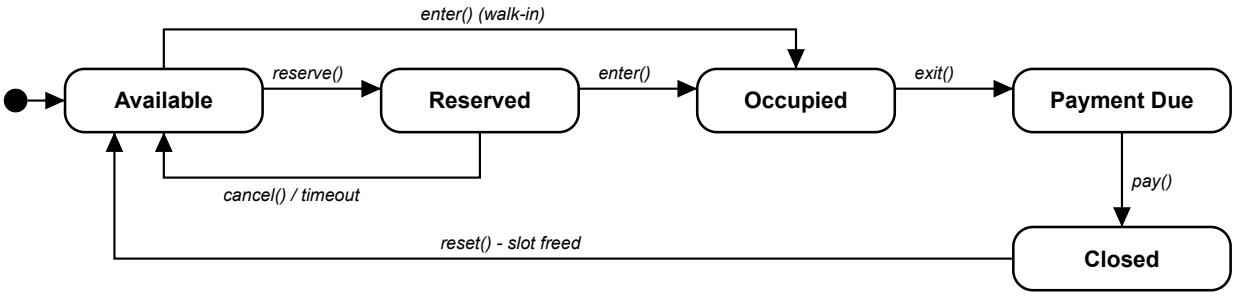
Rule of thumb: long chains of `if (state == ...)` ⇒ State pattern. **State vs CoR:** State = context delegates to *its current state*, states change the context’s state (“how should I behave now?”); CoR = request moves through a *sequence of handlers* (“who will handle this?”). **State vs Strategy:** same structure, but strategies are chosen by the client and don’t switch themselves; states switch automatically.

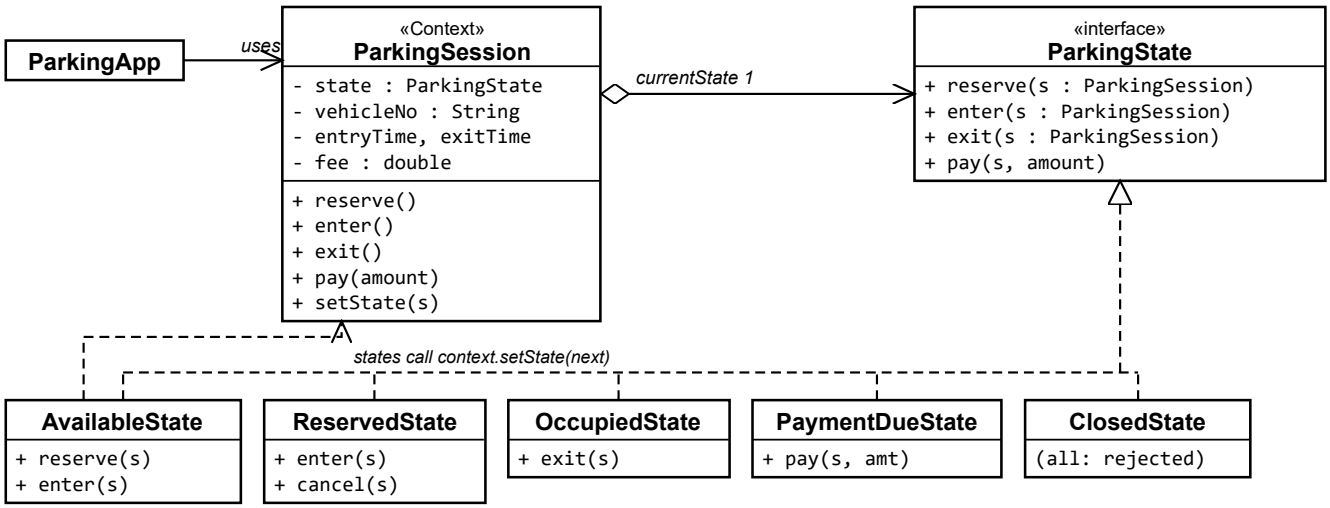
Questions & model answers

In this section: **Smart parking** STATE · **Document workflow** STATE · **Kiosk states** STATE

STATE **SMART PARKING** — Smart parking: session moves Available → Reserved → Occupied → Payment Due → Closed; behaviour depends on state. Why State (4)? State diagram (4). UML (6). Java for the context class (3). *Quiz 1*

Why State: a parking session has a fixed set of well-defined states and the **same action behaves differently** in each (e.g. `enter()` is allowed when Reserved, rejected when Closed; `pay()` only valid in Payment Due). Without the pattern every method needs a big if/switch on the status that grows with each new state. With State, each state’s rules live in one class, transitions are explicit and controlled, invalid actions are rejected cleanly, and new states (e.g. *Overstay*) are added as new classes (OCP).



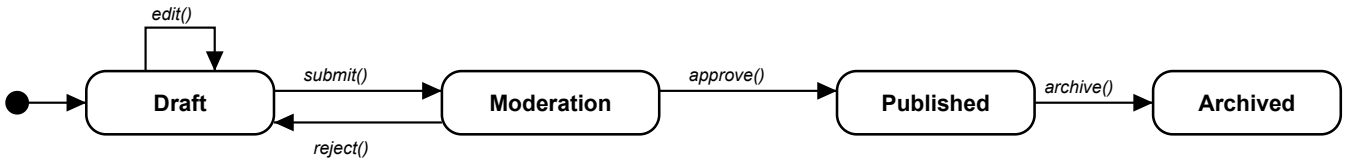



```
public interface ParkingState {
    void reserve(ParkingSession s);   void enter(ParkingSession s);
    void exit(ParkingSession s);      void pay(ParkingSession s, double amount);
}
public class ParkingSession {                                   // CONTEXT
    private ParkingState state = new AvailableState();
    private String vehicleNo; private LocalDateTime entryTime, exitTime; private double fee;

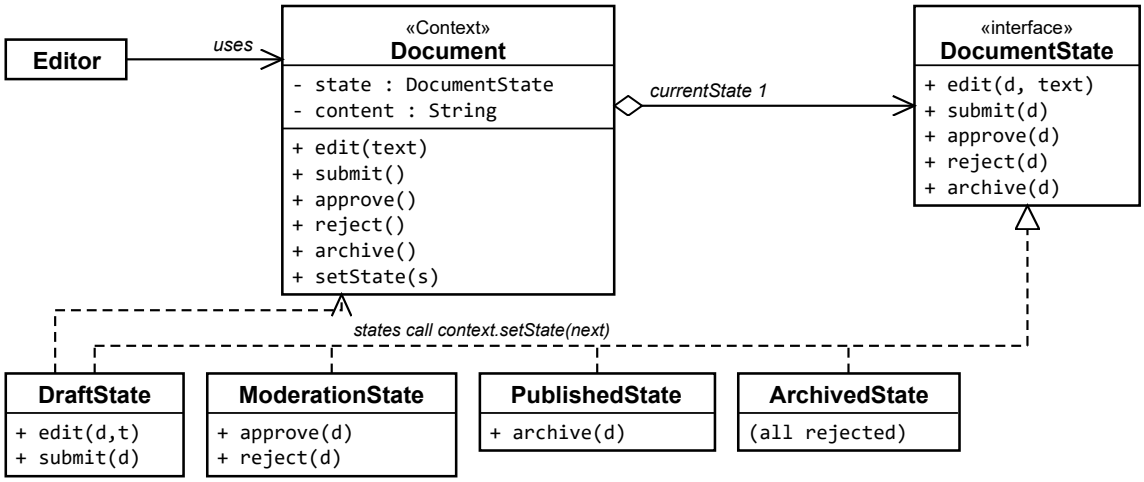
    public void setState(ParkingState s) { this.state = s; System.out.println("-> " + s.getClass().getSimpleName()); }
    public void reserve()          { state.reserve(this); }   // delegate every request
    public void enter()            { state.enter(this); }
    public void exit()             { state.exit(this); }
    public void pay(double amount) { state.pay(this, amount); }

    void recordEntry() { entryTime = LocalDateTime.now(); }
    void recordExit()  { exitTime = LocalDateTime.now();
                         fee = Duration.between(entryTime, exitTime).toHours() * 100; }
    double getFee() { return fee; }
}
public class OccupiedState implements ParkingState {
    public void reserve(ParkingSession s) { System.out.println("Slot occupied"); }
    public void enter(ParkingSession s)   { System.out.println("Already parked"); }
    public void exit(ParkingSession s)    { s.recordExit(); s.setState(new PaymentDueState()); }
    public void pay(ParkingSession s, double a) { System.out.println("Exit first"); }
}
public class PaymentDueState implements ParkingState {
    private final PaymentProcessor processor = new LegacyPaymentAdapter();   // Adapter (Quiz 1 Q5)
    public void pay(ParkingSession s, double a) {
        if (a >= s.getFee() && processor.pay(a)) s.setState(new ClosedState());
        else System.out.println("Payment failed");
    }
    public void reserve(ParkingSession s) { System.out.println("Pay first"); }
    public void enter(ParkingSession s)   { System.out.println("Pay first"); }
    public void exit(ParkingSession s)    { System.out.println("Pay first"); }
}
// AvailableState: reserve -> Reserved, enter -> recordEntry + Occupied. ReservedState: enter -> Occupied.
// ClosedState: every action rejected ("Session closed").
```

STATE DOCUMENT WORKFLOW — Document workflow: Draft (editable → Moderation), Moderation (approve → Published, reject → Draft), Published (no edits → Archived only), Archived (final). State diagram (5); best pattern & reasons (5); class diagram (10); use of interfaces (5). *2024 Q3 (25)*



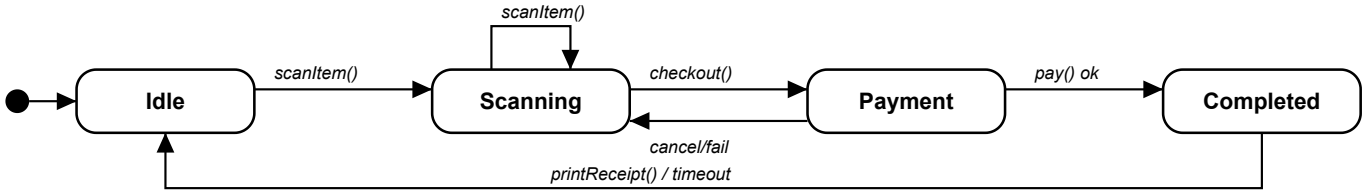
Archived is the final state (no outgoing transitions; draw ◉ after it). Invalid events (edit when Published, approve when Draft) do nothing / show an error.



State pattern: four finite states, each allows different actions and fixed transitions; avoids if/else on a status field in every method; each state class enforces its own rules (Published.edit() refuses). **Interface use:** `DocumentState` is the **contract** — every state must implement all events, so the `Document` can call `state.submit(this)` without knowing the concrete state (**polymorphism**, programming to an interface); new states plug in without changing `Document` (OCP, DIP); states are interchangeable at runtime; easy to mock for testing.

```
public class PublishedState implements DocumentState {
    public void edit(Document d, String t) { System.out.println("Published documents cannot be edited"); }
    public void submit(Document d)  { System.out.println("Already published"); }
    public void approve(Document d) { System.out.println("Already published"); }
    public void reject(Document d)  { System.out.println("Cannot reject a published document"); }
    public void archive(Document d) { d.setState(new ArchivedState()); }
}
// ModerationState: approve -> Published, reject -> Draft; DraftState: edit allowed, submit -> Moderation
```

STATE KIOSK STATES — Self-checkout kiosk states Idle → Scanning → Payment → Completed: incorporate using a pattern and give an example transition. *2025 Q1.1(a), 1.4 (5)*



State: `Kiosk` (context) holds `KioskState`; `IdleState`, `ScanningState`, `PaymentState`, `CompletedState` implement `scanItem()`, `checkout()`, `pay()`, `cancel()`. **Example transition:** in `ScanningState`, the customer presses Checkout → `checkout(kiosk)` computes the subtotal (decorated items' `getCost()`), adds tax via `TaxCalculator.getInstance()` (Singleton) and calls `kiosk.setState(new PaymentState())`. In `IdleState`, `pay()` is rejected ("scan items first").

```
public class ScanningState implements KioskState {
    public void scanItem(Kiosk k, CheckoutItem i) { k.addItem(i); }                // stays Scanning
    public void checkout(Kiosk k) {
        double total = TaxCalculator.getInstance().applyTax(k.subtotal());
        k.setTotal(total);
        k.setState(new PaymentState());                                             // transition
    }
    public void pay(Kiosk k, double amt) { System.out.println("Press checkout first"); }
}
```

Mistakes: context still containing `if (state == ...)` · state diagram without initial state / event labels · forgetting `setState()` in the context · drawing association from states to the interface instead of realization · confusing with CoR or Strategy · transitions to states not in the scenario.